

Ticket To Ride London Design Document

Authors:

Alan Sebastian
Scott Nguyen
Neil Pereira

Team: SNA Solutions

Document Revision History

Date	Version	Description	Author
10/25/2026	0.0	Initial draft	
05/04/2026	0.1	Added design rationale	Neil Pereira, Alan Sebastian, Scott Nguyen
05/04/2026	0.2	Added class diagrams	Scott Nguyen

1 Introduction

This document presents the reasoning behind the digital implementation of *Ticket to Ride London*, outlining the system structure, gameplay mechanics, and key design decisions derived from the official board game rules. The purpose of this document is to provide a clear and structured overview of how the system is implemented

The core problem addressed in this project is the translation of a complex, rule-based physical board game into a consistent and well-defined digital system. *Ticket to Ride* involves multiple interacting components, including players, routes, cards, and scoring systems, all governed by strict rules and constraints. Modelling these elements in software requires careful consideration to ensure that game logic is enforced correctly, player actions are limited appropriately, and the overall gameplay flow remains faithful to the original experience.

The intended audience includes developers, tutors, and other stakeholders involved in the project. It serves as both a technical reference for implementation and a justification of the design choices made throughout the development process.

The document is organised into several sections. It begins with a team introduction, followed by iterations of the class diagram, which represent the design of the game in an object oriented style. This design is reinforced through the rationales presented, explaining key design considerations linked to object oriented principles, design patterns and relationships.

2 Team Information

2.1 Team Name and Membership

The team name is SNA Solutions.



Left to right: Neil Pereira, Scott Nguyen, Alan Sebastian

Name	Contact Details	Strengths	Fun Fact
Scott Nguyen (Sprint 2 Lead)	Email - sngu0065@student.monash.edu Discord - dodgyscott	Technical - Proficient at Java, Python and Javascript - Familiar with OOP design principles Professional - Effective communicator - Strong organisation skills - Always learning	I can drive a forklift
Alan Sebastian	Email - aseb0007@student.monash.edu Discord - alansebastian	Technical - Proficient at Java and Python - OOP design principles Professional - Strong organisation and teamwork skills - Communicates effectively	I can solve a rubix's cube
Neil Pereira	Email - nper0041@student.monash.edu Discord - pereira_neil	Technical - Proficient at Java, Python and C++ - OOP design principles Professional - Strong problem-solving skills - Receptive to feedback and continuous improvement	Im a black belt in karate

2.2 Team Schedule

The team meets once a week in class on Wednesdays at 10-12 am. If for any reason the team is unable to meet during that time, then the discussion is held online via the discord group chat.

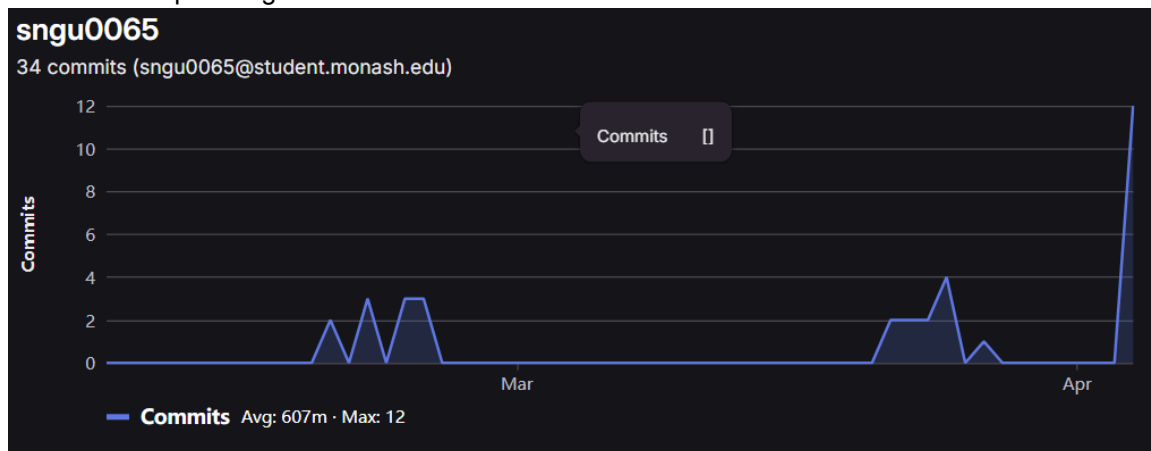
The team has agreed upon a work schedule built on trust, whereby each member can do the work at their own pace as long as it gets done by a date and time agreed by each member. Dates and times when work is expected to be finished are usually decided during each week's meeting.

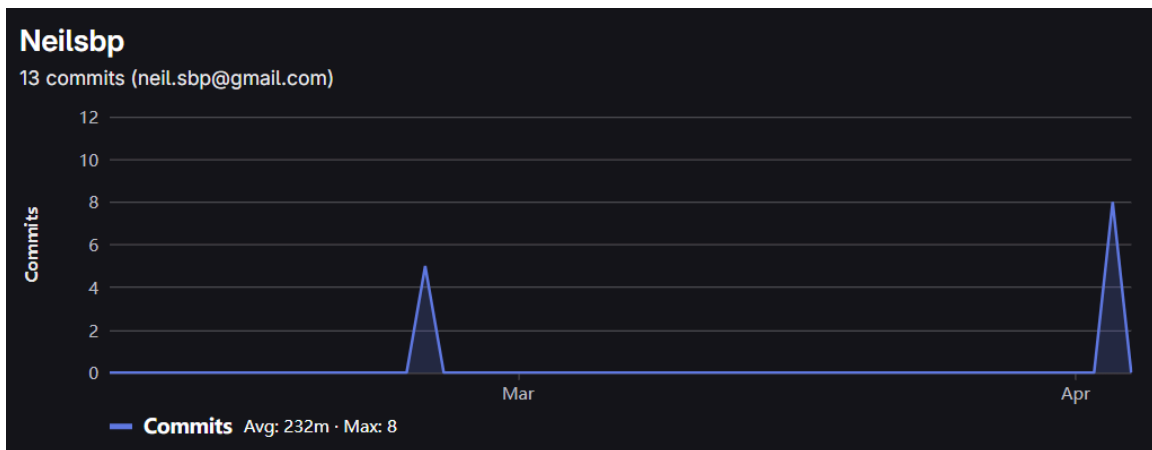
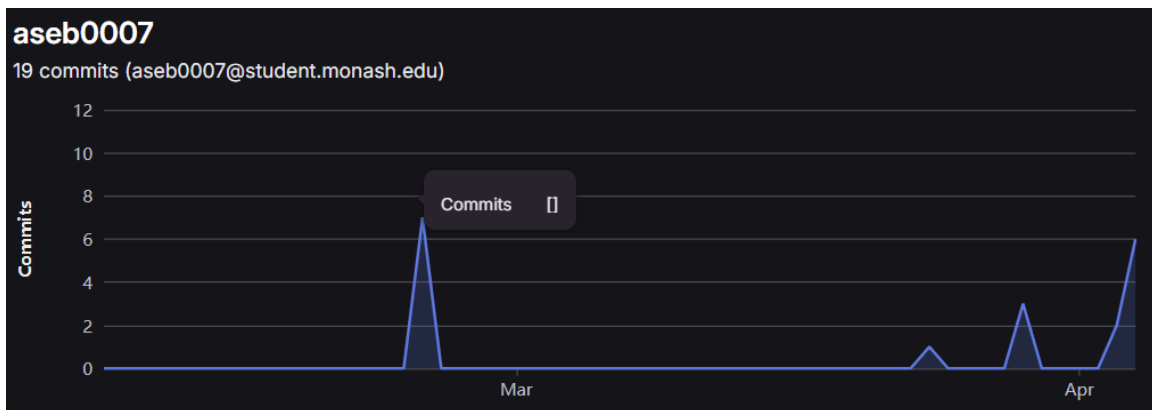
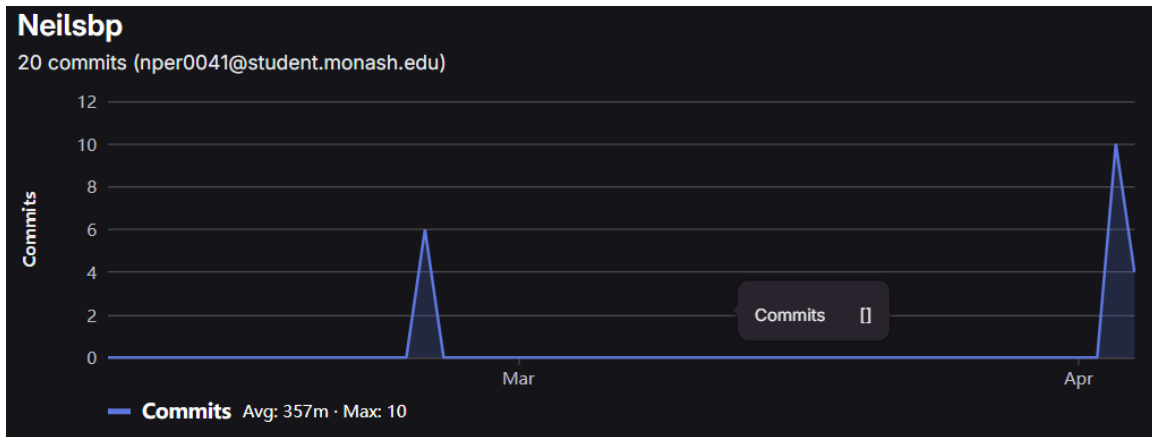
For Sprint 2, each member was assigned to code different parts of the Ticket To Ride game. The split was decided based on estimates of the workload that it entailed. The split we came to was first everything to do with the decks like shuffling, drawing and dealing. Then after that came the main game loop and UI. Then the rest of the game logic like ending the game, claiming routes and checking the longest continuous path. The team regularly communicates via the dedicated Discord channel, where issues are raised, members do regular check-ins, and questions are asked. It was also agreed upon that members would handle all documentation related to their respective parts. For example the member coding the decks would first plan their part in the class diagram. This ensured that all members were aware of the dependencies each other's parts had on the others, and helped to reduce any potential merge conflicts down the line.

During regular communication in Discord, if an issue is sufficient enough to warrant further investigation (that could impede progress), the team has agreed upon ad hoc meetings held as soon as possible after the issue is raised. Not all team members are required to attend these ad hoc meetings, as it is understood that some members are busy. So it was unanimously decided that at least two of the three members must be present at these meetings.

2.3 Team Contribution

*Neil used his private git account for contributions as well





2.4 Link to the Gitlab repository

*Neil used his private git account for contributions as well

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/CL_Wednesday10AM_04

3 Object-Oriented Design

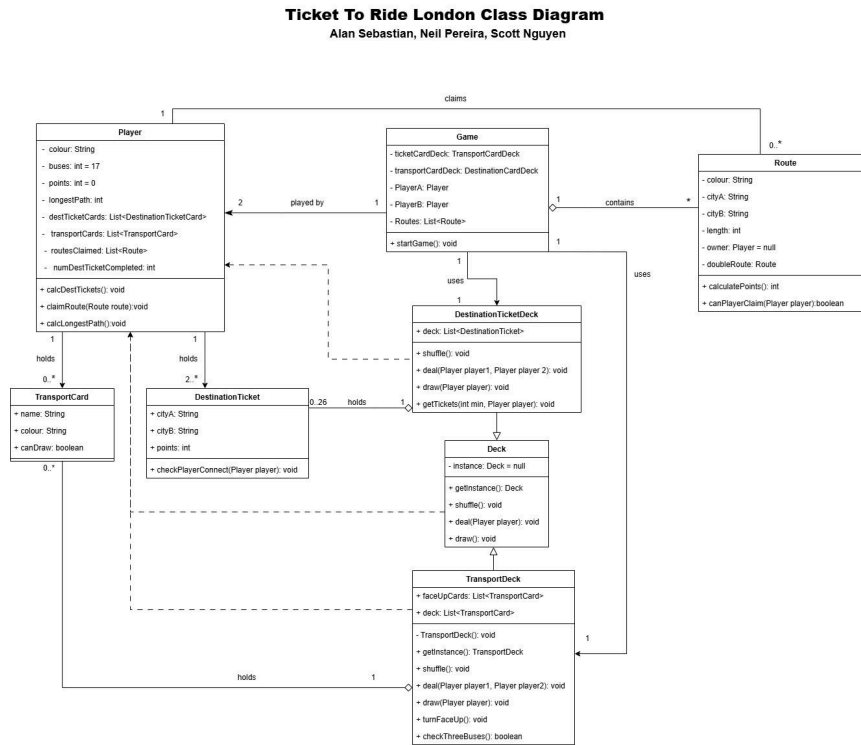
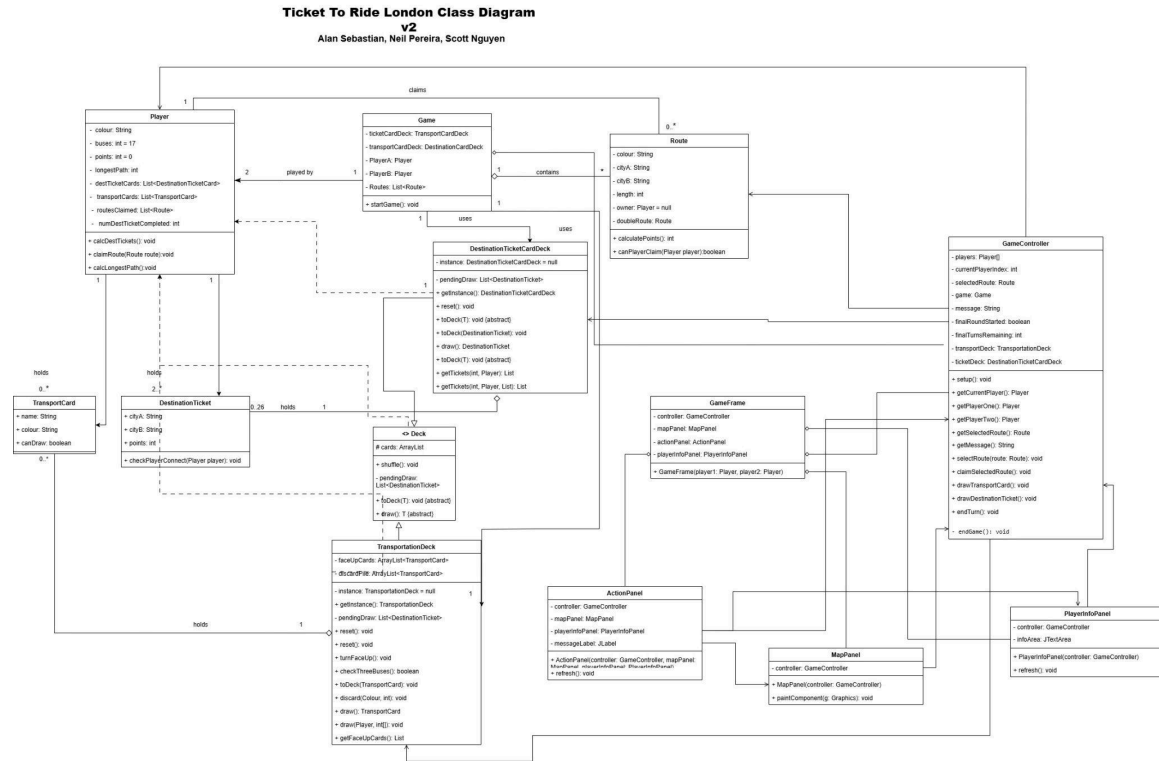


Figure 1.1 - Initial class diagram focussed on game logic.



4 Design Rationales

The DestinationTicket class was created to encapsulate the behaviour of Destination Ticket Cards in the Ticket To Ride game, the Route class for the routes between cities and the Player class for the Players of the game. They were made classes, as they are all distinct, individual entities that exist in the game and have their own attributes and own actions they can perform. For example, Players have the ability to claim routes and calculate longest path, whereas Routes are responsible for checking if a Player can claim it. If they were methods, it would result in overly complicated and lengthy data types. For instance, the Player class stores a list of destination ticket cards, a hashmap of transportation cards, a hashmap of claimed routes etc. In addition, it would be very hard to extend, resulting in lengthy modifications, breaking the Open-Closed principle.

One key relationship in the class diagram was the aggregation between Game and Routes. In the actual implementation, this is between MapPanel and Route. This was an aggregation rather than composition as Routes can exist independently of the map. For instance, Player stores Routes they have claimed to be able to calculate their longest path. This also means there is loose coupling between the two, making the design more flexible.

Both `TransportationDeck` and `DestinationTicketCardDeck` inherit from the abstract class `Deck`. This was done because both have shared behaviour, like `shuffle()` and both store

an array of cards. The main benefit of this in our design is reducing repeated code, adhering to the DRY principle. In other designs, inheritance is important for substitution but our current design does not use these decks interchangeably. Still, this was done with extension in mind, so they can still be used interchangeably if needed, adhering to the Open-Closed Principle.

While the decks inherit from a common deck class, the two card types, Destination Ticket Card and Transportation Cards do not have common inheritance. This is because both serve different purposes, and have nothing common between them besides the fact that they are both held by the player. While our implementation could have had both these classes inherit from a Card class, with Player holding a list of Card[], this would not be efficient nor adhere to SRP. This Card[] data structure would be responsible for many things that could be delegated to two separate data structures like the ones we have in our current implementation. Our final implementation does not have Player store Transportation Cards, instead, it keeps a counter of the colours via hashmap. The DestinationTickets are held in an array. This is better, as it separates the card types into different data structures that assist with their individual responsibilities. Like Deck, we did consider still making a Card class in case future extensions require it, but looking at the current state of both card types, they have nothing in common so it would be useless. If we do need a new abstract class or interface in future, it would be easy to extend, adhering to the Open-Closed principle.

We also made the City's an enumeration rather than a Class by itself. The main reasoning is because it would end up being a data holder class with just one attribute, city name. As seen in the class diagram, cities were originally planned to be strings but during implementation, we realised that enumerations would be better. This is mostly to ensure that misspelled names do not break the code, as it enforces particular spellings of each city, making it easier to refactor. Enums are also good for type safety, preventing invalid cities and catching them early. Maintainability is made easier as all valid values are centralised. This helps with keeping the design close to the Open-Closed principle, as new cities can be added easily. The same reasoning applies to the Colour enumeration.

The game we created is a version of the Ticket To Ride London game, and so, it would have a fixed amount of Routes. However, the cardinality we chose was *. This was because, while the game has a fixed amount of routes, it can realistically be anything. The cardinality represents the upper limit of routes present in the game, and with our current implementation, it is easy to continuously add routes. This adheres to the Open-Closed Principle, and so this is why it has a 'many' cardinality.

Both TransportationDeck and DestinationTicketCardDeck are implemented using the Singleton pattern, ensuring that only one instance of each deck exists for the duration of a game. This is done by making the constructor private, holding the sole instance in a

private static field, and exposing it exclusively through a `getInstance()` static method that creates the instance on first call and returns the existing one on every subsequent call. The game must have exactly one shared draw pile for each card type, if two components independently instantiated a deck, face up state and draw pile counts would diverge, producing illegal game states. The singleton guarantees a single authoritative instance without requiring every collaborator to pass a deck reference down through constructors. A `reset()` static method is provided on each to allow the singleton to be torn down cleanly between games, which avoids the common singleton pitfall of stale state across test runs.

The Strategy design pattern was considered when designing the `calcLongestPath()` method as well as the `calcDestTickets()` method as they were similar in terms of finding connections between two cities. However, we felt that this was over-engineering as there are only two algorithms and so the additional classes and interfaces would have been unnecessary. The possibility of extensions requiring the same algorithms was considered, however, for this sprint we wanted to keep it simple and if further extensions do need this pattern, we can always add it in later.

In the Player class, it was decided that the Transportation Cards be held as a hash map rather than an array list as it was planned in the class diagram. This made it easier to check the amount of each card a Player had for things like claiming routes. If it were a list, the Player class would need to constantly iterate and count the number of a particular card each time it needs to. This also adheres to the Single Responsibility Principle, as its hashmap data structure is responsible for keeping count of each colour, not the Player.

Similarly, the claimed routes of the player were kept as an adjacency list via hashmap as opposed to a list like the class diagram. This was because during implementation, it would have been unnecessarily complicated to find paths between cities when checking if a player had completed a Destination Ticket card or not. It would have resulted in lots of repeated code when scanning through the list. This is why an adjacency list was better, as it not only simplified implementation, but allowed for modular code. For example, giving us the benefit of being able to use Breadth-First Search.

In addition, encapsulation was utilized to separate responsibilities and prevent direct access to other classes' attributes. For instance, getter and setter methods were prevalent in our code (omitted from the diagram for simplicity), as they are simple ways to protect attributes from being directly accessed. Separation of concerns was demonstrated through the `canPlayerClaim()` and `claimRoute()` methods of Route and Player classes respectively. While both could have been combined to form one method, the team felt it was better to separate them as they represented different responsibilities. The reason `canPlayerClaim()` was placed in Route was because whether or not a player can claim the route is if their cards match the route's properties.

The user interface was designed with a strong focus on separation of concerns, clarity and maintainability. The UI was divided into distinct components, namely the MapPanel, PlayerInfoPanel and ActionPanel, each responsible for a specific aspect of the interface. The MapPanel handles rendering the game board and route selection, the PlayerInfoPanel displays player related information such as scores and current turn and the Action Panel manages all user interactions through buttons. This structure follows the Single Responsibility principle, ensuring that each component has a clearly defined role and can be modified independently without affecting other parts of the system.

A key design decision was to centralise all game logic within the GameController rather than allowing UI components to directly manipulate the game state. All user actions trigger through the UI such as drawing cards or claiming routes are passed to the controller, which processes the logic and updates the state accordingly. The UI then refreshes to reflect these updates. This approach ensures consistency, reduces the risk of errors between the frontend and backend, making the system easier to test and extend.

The interface follows an event driven model using action listeners, which aligns naturally with Java Swing's architecture. Each button in the ActionPanel triggers a specific action in the controller, followed by a refresh of the UI. This ensures that the application remains responsible and that each user interaction is handled in a clear and modular way. The use of an explicit refresh method was a deliberate choice to keep the update logic simple and predictable and avoid the added complexity of more advanced patterns such as observers or data binding.

User feedback was also an important consideration in the design. A message label is used to display real time feedback to the player after each action, such as confirming successful moves, indicating invalid actions or showing turn changes. This helps improve usability by ensuring that each player always understands what is happening in the game and what their next step should be. Additionally, simple layout managers such as FlowLayout and BorderLayout were used to keep the interface clean and easy to understand, prioritising functionality and readability over complex styling.

Overall, the UI design balances simplicity and functionality, providing a modular and maintainable structure that integrates effectively with the game logic while delivering a clear and responsible user experience.

5 Supplementary Documentation